IA Assignment 2

Naïm Saadi Gallego Eduard Térmens Botanch

Decision Trees

For this implementation, we created an iterative tree building algorithm using a stack. We also implemented categorical values as subsets.

We implemented **cost-complexity pruning**[1] to reduce overfitting. The pruning method compares the error increase from converting a node to a leaf against the `prune_threshold` parameter. We also use the `beta` parameter to require a minimum information gain before making a split. We tested different combinations as shown in Figure 1. We observed that setting `beta` to 0 gave us the best results.

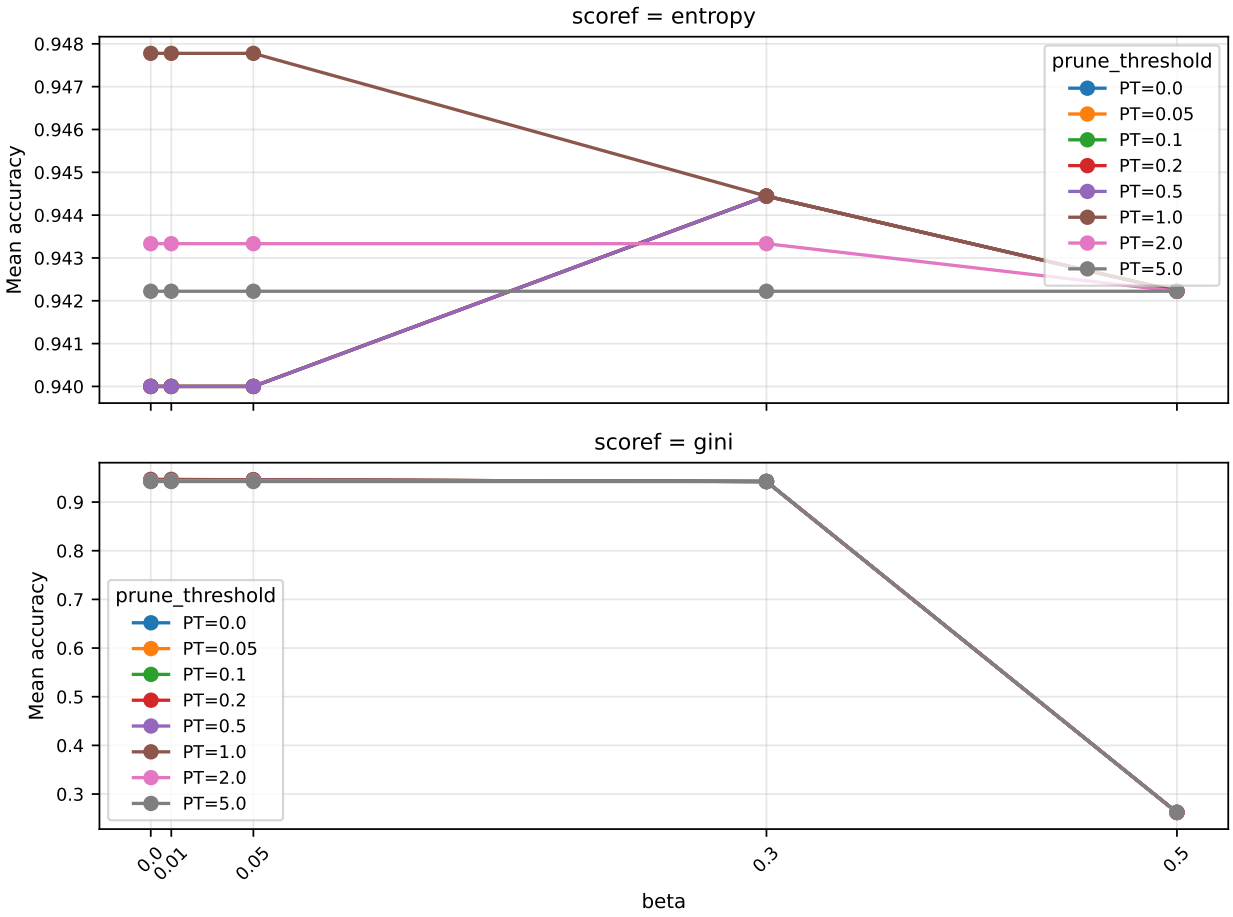

Figure 1: Mean accuracy by beta prune_threshold and scoref in the iris dataset

The best hyperparameter combination found was score function entropy, $\beta = 0.0$, prune threshold = 1.0, achieving a mean score of 94.78%.

Naive Bayes

For this implementation, we decided on tokenizing the message by searching if it contained some features like currency symbols, phone numbers[2], *urls* or all-caps words. We also check if any of the words are common english stopwords[3]. After that, we return the words of the message in lowercase and the features.

We first struggled at understanding what data from the training set we had to store, but with the help of some sources[4], we finally decided on storing the essential: the times a feature appears in a document (for each class), the total of features that are spam, the total features that are spam and the prior probabilites for each class.

We then started trying different parameters, adn the one that had the biggest impact was the **assumed probability**, which we started tweaking with different values as shown in Figure 2.

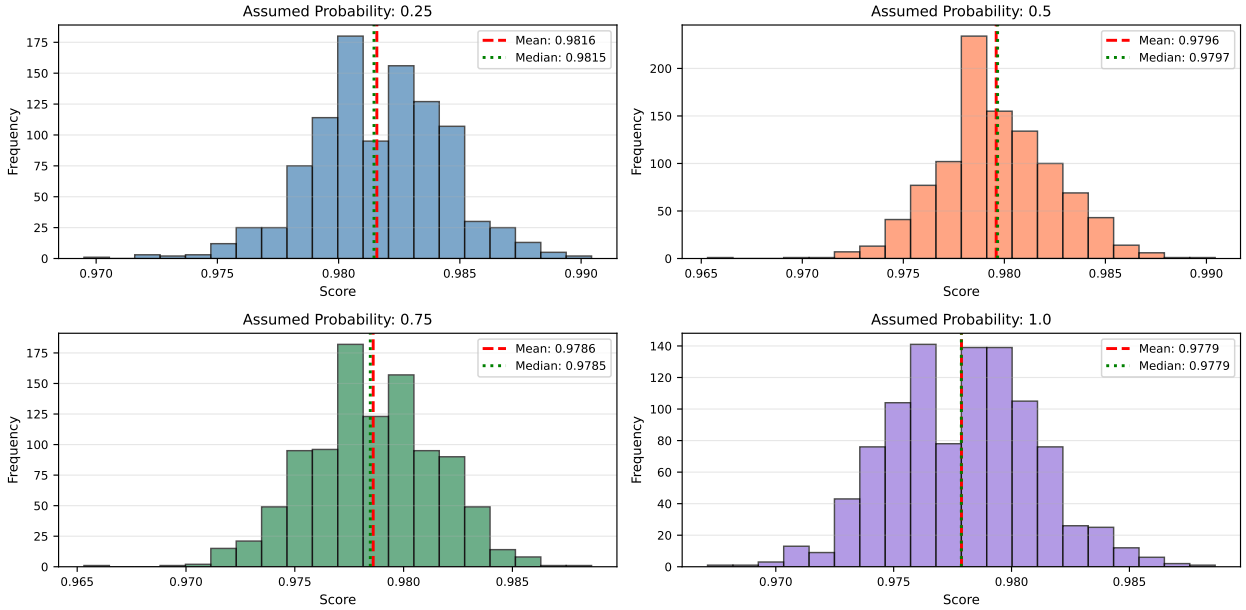

Figure 2: Score across 1000 iterations for different assumed probabilities

We overall have very positive scores, being an assumed probability of 25% the best with a mean of 98.16% and a median of 98.15%.

K-means

Design choices and problems found

For this implementation, we decided to create a class called `CentroidManager`, which generates the starting centroids and is also used to compute the positions of the new centroids. We also used the `rich` module to debug our code and to show the output.

A critical bug that we had was storing the closest centroid but not the closest distance, just the last one computed. That caused the total distance to increase with the centroids, something mathematically impossible.

Results and the elbow method

As we can see in Figure 3, the graph shows the total distance decreasing when we increase the value for k, having more centroids and clearer data. We decided to use the **squared euclidean** metric for its efficiency (since it doesn't need to compute the square root). It was also useful to use the *elbow method*.

The *elbow method*[5] suggests that, if we plot k vs WCSS *(Within-Cluster Sum of Squares)*, there is a point where increasing the value for k doesn't have a meaningful impact, which can lead to **overfitting**. As we can see in Figure 3, at $k = 5$ the distance decreases very slowly, being this a good value for k. It is also important to note that $k = 5$ is a *local optimum*.

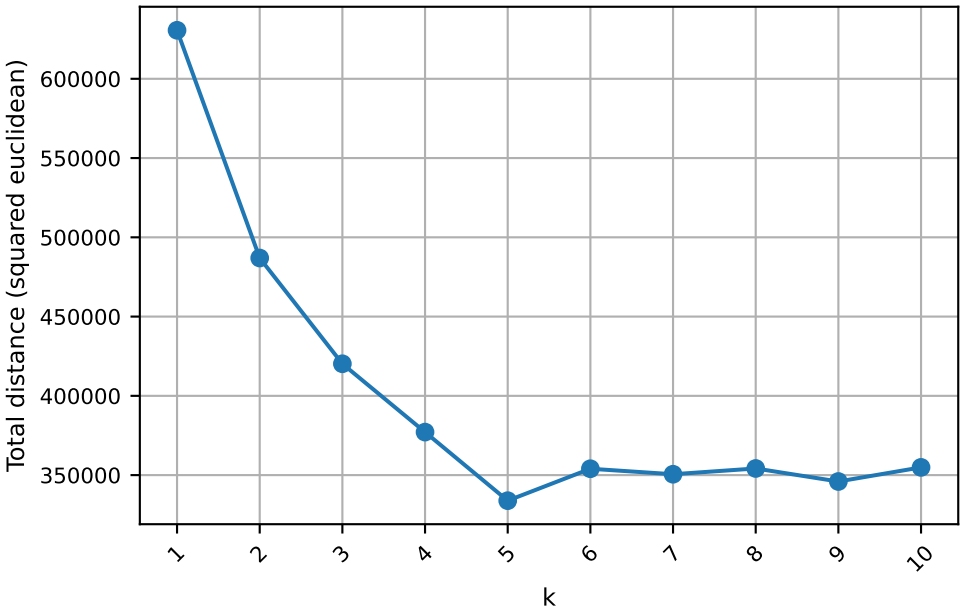

Figure 3: Total distance depending on k

References

[1] A. K. Singh. "How to choose α in cost-complexity pruning? "[Online]. Available: https://www.geeksforgeeks.org/machine-learning/how-to-choose-a-in-cost-complexity-pruning/

[2] R. K. Thapliyal. "Answer to: 'regular expression to match standard 10 digit phone number'. "[Online]. Available: https://stackoverflow.com/a/16699507

[3] S. Bleier. “Nltk’s list of english stopwords. ”[Online]. Available: https://gist.github.com/sebleier/554280

[4] J. Starmer. “Naive bayes, clearly explained!!! ”[Online]. Available: https://youtu.be/O2L2Uv9pdDA?si=r17_b3JoPNkc50fJ

[5] A. Gupta. “Elbow method for optimal value of k in kmeans. ”[Online]. Available: https://www.geeksforgeeks.org/machine-learning/elbow-method-for-optimal-value-of-k-in-kmeans/